

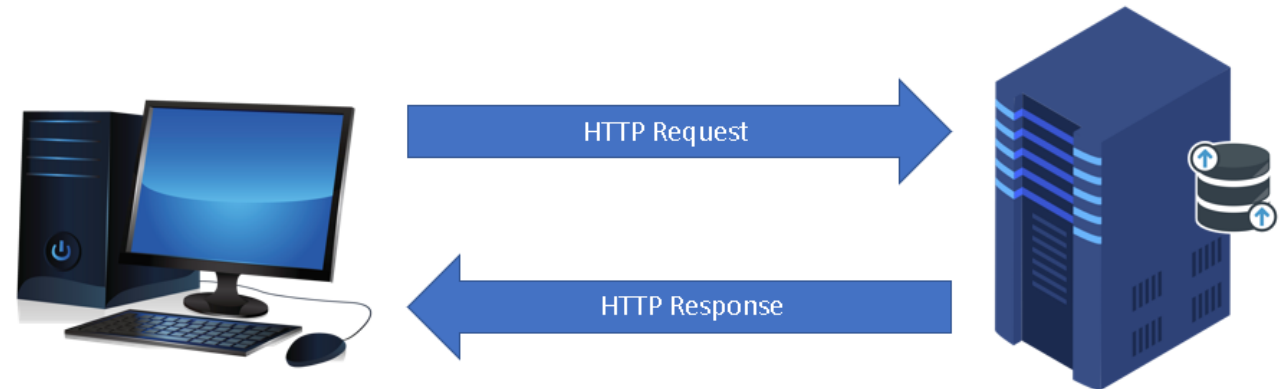
Protocolo HTTP

Hypertext Transfer Protocol

HTTP

HTTP

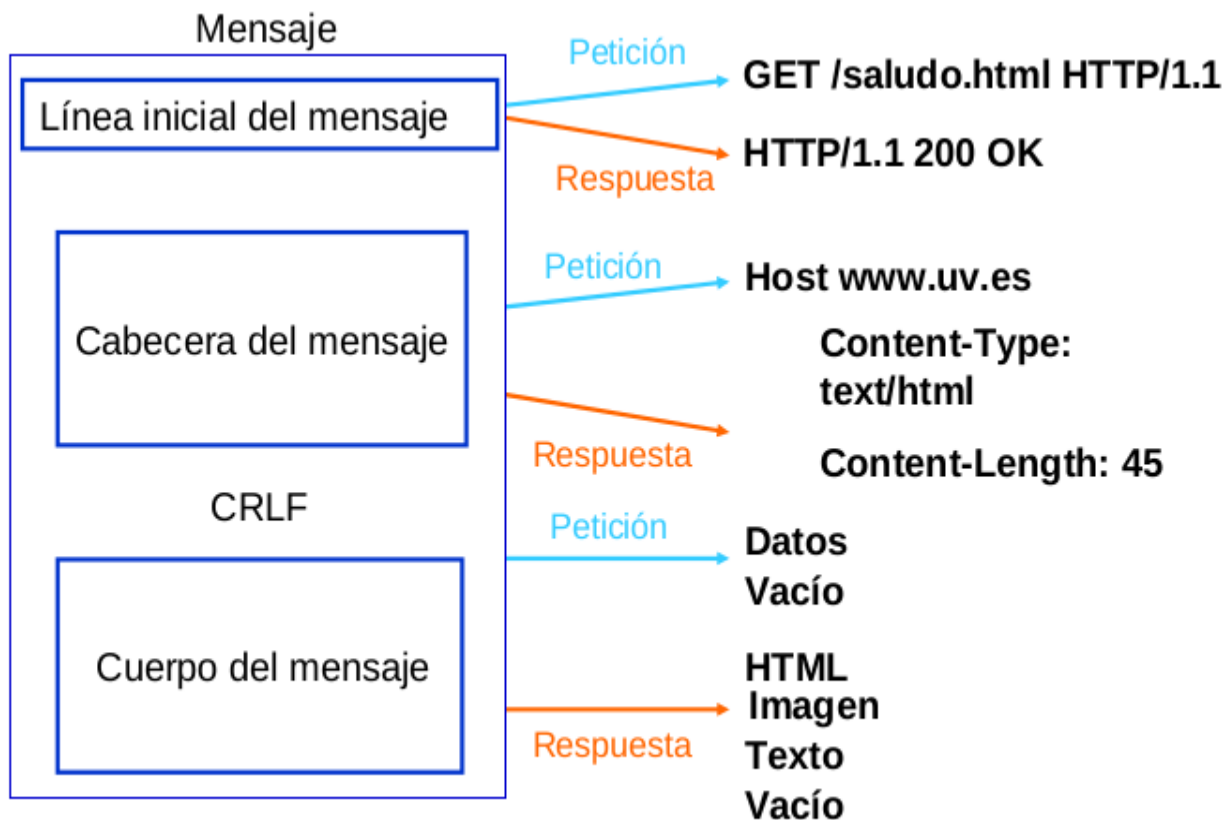
- El protocolo HTTP (*Hypertext Transfer Protocol*) es el protocolo de aplicación fundamental para la transferencia de datos en la World Wide Web, permitiendo el intercambio de documentos HTML, imágenes y recursos entre un cliente (navegador) y un servidor
- Funciona bajo un modelo petición-respuesta
- Es un protocolo sin estado (stateless)
- Utiliza el puerto 80 por defecto (HTTPS utiliza el 443)
- Es un protocolo de la capa de Aplicación



MODELO TCP/IP (Transmission Control Protocol/Internet Protocol)

APLICACIÓN	Protocolos de usuario. Intercambio de datos entre aplicaciones.	HTTP SMTP TELNET FTP DNS SNMP
TRANSPORTE	Puerto de Red. Intercambio de datos. Control de errores, segmentación.	TCP UDP
RED	Enrutamiento a través de la dirección IP. Conexión con diferentes redes.	IP
CAPA DE ENLACE	Conexión física. Conoce la red local (MACs)	Ethernet, Token Ring, etc.

Request - Response



- **Petición (Request):** El cliente envía una petición al servidor para recuperar un recurso específico. La petición incluye información como el método HTTP seleccionado (GET, POST, etc.), la URL del recurso y los encabezados HTTP necesarios.
- **Respuesta (Response):** El servidor recibe la petición y responde con la información solicitada, junto con un código de estado HTTP (Ejemplo: 200 OK o 404 Not Found) y encabezados adicionales. La respuesta puede incluir datos en HTML, imágenes, archivos JSON, etc.

Request - Response

• Request

```
1 GET /index.html HTTP/1.1
2 Host: www.mi-sitio.com
3 User-Agent: Mozilla/5.0
4 Accept: text/html
```

• Response

```
1 HTTP/1.1 200 OK
2 Date: Wed, 2 Oct 2024 18:45:00 GMT
3 Server: Apache/xx.0
4 Content-Type: text/html
5 Content-Length: 5059
6
7 <!DOCTYPE html>
8 <html>
9 <head>
...

```

GET: método HTTP utilizado para solicitar un recurso.

/index.html es la ruta del recurso que se está solicitando en el servidor.

HTTP/1.1 indica la versión del protocolo HTTP que se está utilizando.

Host: www.mi-sitio.com especifica el nombre del host al que se hace la petición.

User-Agent: Mozilla/5.0 proporciona información sobre el agente de usuario o el navegador que realiza la petición.

Accept: text/html indica que el cliente prefiere recibir el recurso en formato HTML.

HTTP/1.1 200 OK: indica que el recurso solicitado se encontró y se entregó correctamente.

Date: Wed, 2 Oct 2024 18:45:00 GMT : proporciona la fecha y la hora en que se generó la respuesta.

Server Apache/xx.0: indica el software del servidor web y su versión.

Content-Type: text/html: indica que el tipo de contenido de la respuesta es HTML.

Content-Length: 5059: indica el número de bytes del contenido de la respuesta.

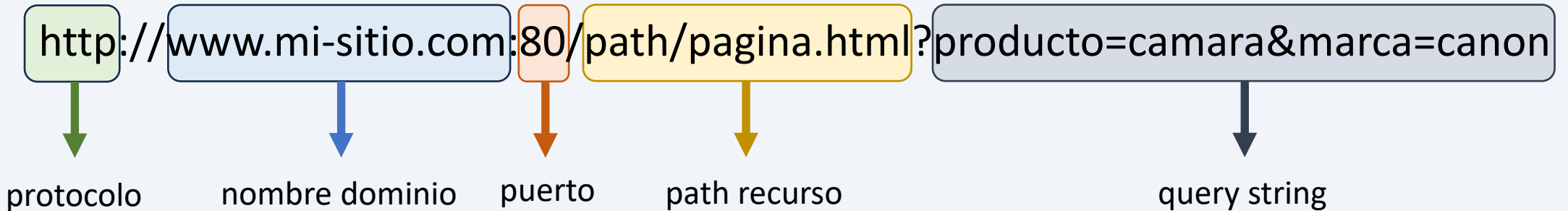
El resto del contenido del mensaje es el documento HTML que ha solicitado el cliente.

URL – Uniform Resource Locator

Es un tipo de URI que indica dónde está el recurso y cómo se accede a él

http://www.mi-sitio.com:/path/pagina.html

http://www.mi-sitio.com:80/path/pagina.html



Métodos (verbos) de petición HTTP

Indican la acción que se desea realizar sobre un recurso determinado

Método			
<i>GET</i>	<i>Solicita un recurso</i>	<i>Cargar una página o pedir datos a una API</i>	<i>Parámetros por query string</i>
<i>POST</i>	<i>Envía datos para crear un recurso</i>	<i>Crear un usuario, enviar un formulario, etc.</i>	<i>Parámetros en el cuerpo (ej. JSON)</i>
<i>PUT</i>	<i>Reemplaza un recurso (o lo crea si no existe)</i>	<i>Actualizar todo un recurso</i>	<i>Parámetros en el cuerpo (ej. JSON)</i>
<i>DELETE</i>	<i>Elimina un recurso</i>	<i>Borrar un registro</i>	
<i>PATCH</i>	<i>Modificación parcial sobre un recurso</i>	<i>Cambiar sólo el email de un cliente</i>	
<i>HEAD</i>	<i>Solicita sólo la cabecera</i>	<i>Comprobar si existe un recurso</i>	

Código de estado HTTP

Entero de 3 dígitos incluido en una respuesta HTTP que comunica el resultado de procesar una solicitud y determina parte de la semántica de la respuesta.

$$C = \{c \in \mathbb{N} \wedge 100 \leq c \leq 599\}$$

INTERVALO	CLASE	SEMÁNTICA GENERAL
$100 \leq c \leq 199$	1xx	Respuesta informativa o provisional
$200 \leq c \leq 299$	2xx	Solicitud procesada satisfactoriamente
$300 \leq c \leq 399$	3xx	Se requiere redirección o alguna acción adicional
$400 \leq c \leq 499$	4xx	La solicitud presenta un problema atribuible al cliente
$500 \leq c \leq 599$	5xx	El servidor no pudo procesar una solicitud aparentemente válida

RESTfull APIs

Representational State Transfer

API REST

Desarrollo de RESTful APIs

REST (Representational State Transfer)

- Semántica HTTP y contratos
- Un API REST no basta que “funcione”, tiene que ser predecible
- El consumidor debe entender:
 - Qué hace
 - Qué resultado obtuvo
 - Si debe reintentar o no
 - Si un error fue propio o del servidor
 - Estructura clara de las respuestas

Semántica HTTP y contratos

Semántica HTTP

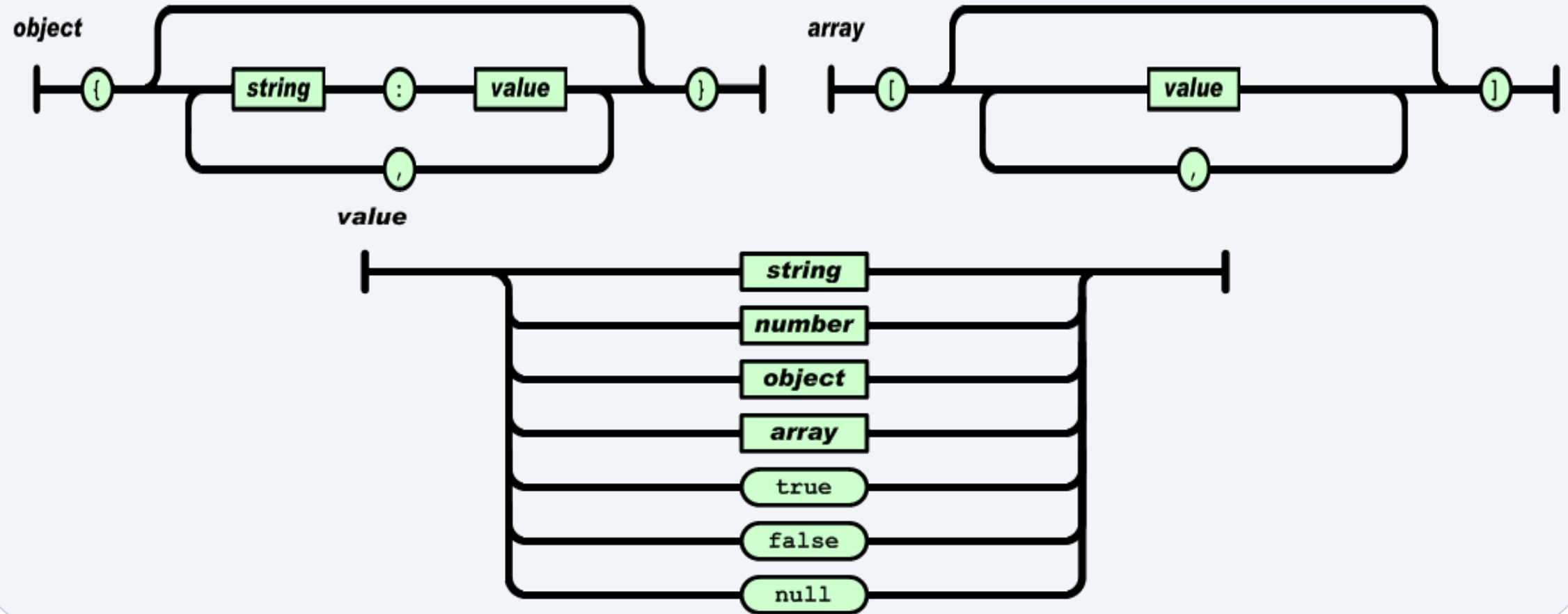
- Uso correcto de métodos HTTP (GET, POST, PUT, PATCH, DELETE)
- Uso correcto de códigos de estado (200, 201, 204, 400, 404, etc)

Contrato

Acuerdo entre productor y consumidor

- Endpoints
- Formato JSON
- Campos obligatorios / opcionales
- Códigos de estado
- Errores esperables
- Reglas de uso

JSON – JavaScript Object Notation



```
{  
  "id": 1,  
  "fabricante": "Samsung",  
  "productos": [  
    {  
      "id": 101,  
      "producto": "Monitor 27 pulgadas",  
      "precio": 249.99  
    },  
    {  
      "id": 102,  
      "producto": "Disco SSD 1 TB",  
      "precio": 89.5  
    }  
  ]  
}
```

- 200 OK (Operación correcta y retorna contenido)
 - GET /products/10
 - PUT /orders/15 (si devuelve el recurso actualizado)
- 201 Created (Se crea un nuevo recurso)
 - Devolver código 201 / cabecera Location con URL del nuevo recurso
 - POST /orders
- 204 No Content (Operación correcta, pero respuesta sin cuerpo)
 - DELETE /products/10
 - PUT /users/7 (si no devuelve cuerpo)
 - PATH /orders/5/cancel (si no devuelve información)

- 400 Bad Request (petición mal formada)
 - JSON inválido, campo obligatorio ausente, tipo incorrecto, etc.
- 404 Not Found (el recurso no existe)
 - GET /products/9999
 - DELETE /orders/89898
- 409 Conflict (el estado actual del recurso impide la operación)
 - Es un conflicto de estado o de negocio
 - Intentar cancelar un pedido ya enviado
 - Intentar registrar un usuario con email duplicado
- 422 Unprocessable Entity (petición bien formada, pero sin sentido semántico)
 - Se usa menos. La mayoría utiliza 400

- 500 Internal Server Error
 - Error inesperado del Servidor
 - Ejemplo:
 - Null pointer
 - Excepción no controlada
- CONSISTENCIA
 - ¡Utilizar siempre de la misma forma los códigos!
 - Misma regla en todos los endpoints
 - Misma semántica en todos los servicios

- La clave no es solo elegir buenos códigos, sino usarlos **siempre igual**
- Errores habituales
 - A veces retorna 404, otras 400 para lo mismo
 - Un endpoint retorna 200 en creación y otro 201
- Diseño correcto
 - Misma regla en todos los endpoints
 - Misma semántica en todos los servicios
 - Documentación clara

Idempotencia: repetir la misma operación una o varias veces produce el mismo efecto final

Clave en sistemas distribuidos ya que hay reintentos, timeouts, etc.

- **GET** debe ser idempotente
 - Se puede ejecutar muchas veces y el estado del sistema no cambia
- **PUT** debe ser idempotente
 - PUT reemplaza estado. Si se reenvía la misma petición, el resultado es el mismo
- **DELETE** debe ser idempotente
 - Borrar varias veces un recurso no debería tener distintos efectos más allá del primero
- **POST “no es idempotente por defecto”**
 - Suele crear nuevos recursos, repetir la misma llamada puede generar duplicados

Idempotencia: repetir la misma operación una o varias veces produce el mismo efecto final

Clave en sistemas distribuidos ya que hay reintentos, timeouts, etc.

- **Idempotency-Key**
 - Para evitar duplicados en operaciones sensibles
 - El cliente genera y envía una clave con la petición
 - El servidor la almacena temporalmente y la asocia a la petición para controlar si se reenvía
 - Ej:
 - Pagos
 - Creación de pedidos
 - Operaciones financieras
 - Reservas
 - Acciones sensibles que el cliente podría reintentar

DTOs – Data Transfer Object

- No exponer las entidades JPA. Representan el “**modelo**” y no el “**contrato**”
- ¿por qué?
 - Se acopla la API al modelo interno
 - Cualquier cambio en la entidad afecta al contrato
 - Se pueden filtrar datos que no se deberían exponer
 - Pueden aparecer problemas con relaciones lazy/eager

• Mapeo

entidad > DTO

DTO > entidad

- Manual
- MapStruct

```
import java.math.BigDecimal;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

public record CreateProductRequest(
    @NotBlank String name,
    @NotNull @Positive BigDecimal price
) {}
```

```
import java.math.BigDecimal;

public record ProductResponse(
    Long id,
    String name,
    BigDecimal price
) {}
```

- Se incorporaron inicialmente como previa en la versión JDK 14, y posteriormente movida a estable en el JDK 17.
- Se trata de una implementación de las llamadas data-classes o el patrón Data Transfer Object (DTO).
- Por lo general únicamente especificaremos qué atributos nos interesa que tenga un registro, y al compilar un registro, el compilador se ocupará de generar constructores, getters y métodos como `toString()`, `equals()` o `hashCode()` de forma automática.
- Permiten generar data-classes inmutables con muy poco código.

```
public class ClieteDto0 {  
    private final Long idCliente;  
    private final String nombre;  
    private final String email;  
  
    public ClieteDto0(Long idCliente, String nombre, String email) {  
        this.idCliente = idCliente;  
        this.nombre = nombre;  
        this.email = email;  
    }  
  
    public Long idCliente() {  
        return idCliente;  
    }  
  
    public String nombre() {  
        return nombre;  
    }  
  
    public String email() {  
        return email;  
    }  
}
```

```
@Override  
public int hashCode() {  
    return Objects.hash(email, idCliente, nombre);  
}  
  
@Override  
public boolean equals(Object obj) {  
    if (this == obj) return true;  
    if (obj == null) return false;  
    if (getClass() != obj.getClass()) return false;  
    ClieteDto0 other = (ClieteDto0) obj;  
    return Objects.equals(email, other.email)  
        && Objects.equals(idCliente, other.idCliente)  
        && Objects.equals(nombre, other.nombre);  
}  
  
@Override  
public String toString() {  
    return "ClienteEquivalencia [idCliente=" + idCliente + "...";  
}  
}
```

```
public record ClienteDto(  
    long id,  
    String nombre,  
    String email,  
    int edad,  
    List<String> telefonos  
) {}
```

```
public static void main(String[] args) {  
    ClienteDto cli = new ClienteDto(33L, "Marta", "marta@ej.com", 34, List.of("666777888"));  
    System.out.println(cli);  
  
    System.out.println(cli.edad());  
  
    ClienteDto nuevo = new ClienteDto(33L, "Marta", "marta@ej.com", 34, List.of("666777888"));  
  
    System.out.println(nuevo.equals(cli));  
}
```

Record con validación en el constructor

```
public record Cliente2(Integer idCliente, String nombre, String apellidos) {  
  
    //Podemos hacer esto  
    public Cliente2 {  
        if (idCliente <= 0 || isEmpty(nombre) || isEmpty(apellidos))  
            throw new IllegalArgumentException();  
    }  
}
```